Project Title: FieldLink — A Mobile-First Construction Management Platform

Student: Bridgette Kimotho

Admission Number: 250311

Course: Flutter and Dart

Supervisor: Wilson Ochieng

Institution: Institute of Software Technology

Year: 2026

Declaration

I hereby declare that this project report is my original work and has not been submitted to any other institution for the award of a degree, diploma, or certificate.

Signed: _____ **Date:** _____

Abstract

Construction projects in Kenya, and across many developing regions, suffer from fragmented communication, lost paperwork, poor tool accountability, and delayed worker payments. Existing construction management software is often expensive, desktop-oriented, or unsuitable for sites with unreliable internet connectivity. This project presents **FieldLink**, a mobile-first construction management platform built with Flutter and powered by Supabase, that centralises project tracking, worker task assignment, tool checkout via QR codes, and worker payments — including real M-Pesa B2C integration through Safaricom's Daraja API.

FieldLink supports three roles — **Admin**, **Manager**, and **Worker** — with permissions enforced at the database level using Supabase Row Level Security. The application is fully bilingual (English and Kiswahili), supports light and dark themes, provides offline read-caching for tasks and payments, and uses a separate server-side Node.js service for secure M-Pesa transactions. The system was tested on Android and web, and validated against a series of functional and security scenarios.

Keywords: Construction Management, Flutter, Supabase, M-Pesa, Daraja API, Offline-First, Row Level Security, Kenya.

Table of Contents

1. Introduction
2. Literature Review
3. System Analysis and Design
4. Implementation
5. Testing
6. Results and Discussion
7. Conclusion and Future Work

8. References
9. Appendices

Chapter 1 — Introduction

1.1 Background

The construction industry is a major contributor to Kenya's economy, yet it remains one of the least digitised sectors. Communication typically happens through WhatsApp groups, phone calls, and paper notebooks. Project managers track tasks on spreadsheets or whiteboards. Tool checkouts are rarely logged. Worker payments are handled through cash or manual M-Pesa transfers, often with no consolidated record.

These practices lead to:

- **Communication delays** — updates arrive late or not at all
- **Lost paperwork** — daily logs, incident reports, and receipts go missing
- **Poor task visibility** — managers cannot easily see who is doing what
- **Asset mismanagement** — tools disappear or are double-booked
- **Payment disputes** — no reliable record of what was paid, to whom, and when

Existing software such as Procore, Buildertrend, and Autodesk Construction Cloud addresses these problems but is designed for large firms in developed markets. It is expensive, assumes reliable internet, and lacks integration with African payment systems.

1.2 Problem Statement

Construction teams in Kenya lack an affordable, mobile-first tool that:

1. Works reliably with intermittent internet connectivity
2. Enforces role-based access control without relying on user trust
3. Tracks tools using QR codes
4. Records worker payments in a centralised, auditable way
5. Integrates with M-Pesa, the dominant payment method in Kenya
6. Supports local languages

1.3 Objectives

1.3.1 General Objective

Design and implement a mobile-first construction management platform for Kenyan construction teams that works offline, enforces role-based security at the database level, and integrates with M-Pesa for worker payments.

1.3.2 Specific Objectives

1. Build a Flutter application supporting Admin, Manager, and Worker roles.
2. Design a PostgreSQL database hosted on Supabase with full Row Level Security.
3. Implement project, task, asset, and payment management modules.

4. Integrate Safaricom's Daraja B2C API for worker payments via a secure server-side service.
5. Provide offline caching for tasks and payment history.
6. Support English and Kiswahili with full UI translation.
7. Enforce security so that no payment credentials or service-role keys appear in the mobile application.

1.4 Scope

In Scope

- Three user roles with role-based dashboards
- Authentication via email and password
- Project creation with Kenya county and sub-county selection
- Task assignment and completion
- QR-based asset checkout and check-in
- Payment records for M-Pesa, cash, and bank transfer
- M-Pesa B2C integration via Daraja sandbox
- In-app notifications
- Offline read-caching of tasks and payments
- Bilingual support (English, Kiswahili)
- Light, dark, and system themes

Out of Scope (Future Work)

- Online chat
- Google Sign-In
- Automated email verification via Brevo
- Safety and incident reporting
- Site geofencing and attendance
- Personnel training and certification tracking
- Change orders and budget tracking
- Full offline write synchronisation

1.5 Significance

FieldLink demonstrates that an affordable, secure, offline-capable construction management tool can be built with modern serverless technology and integrated with M-Pesa, the payment method that dominates Kenyan commerce. The project also provides a reference architecture for any mobile application that must keep payment credentials off the client while still providing real-time transactional feedback.

1.6 Methodology

The project followed an **iterative, phased methodology**:

1. **Phase 1** — Flutter foundation, theme, navigation
2. **Phase 2** — Supabase schema, authentication, RLS, storage
3. **Phase 3** — Localisation (English + Kiswahili)
4. **Phase 4** — Admin role
5. **Phase 5** — Manager role
6. **Phase 6** — Worker role

7. **Phase 7** — Assets and QR codes
8. **Phase 8** — Payments
9. **Phase 9** — Notifications
10. **Phase 10** — Offline caching
11. **Phase 12** — Testing and release build

Each phase was tested in isolation before the next began, ensuring that regressions could be traced to the most recent change.

Chapter 2 — Literature Review

2.1 Construction Management Systems

Traditional systems like Procore and Buildertrend offer comprehensive functionality but are designed for enterprise customers in mature markets. They assume:

- Reliable high-speed internet
- Desktop or tablet use
- Integration with banking systems (ACH, wire transfers)
- High subscription costs per user

For a Kenyan construction firm with 10–50 workers, these assumptions rarely hold.

2.2 Mobile-First Design

Recent studies indicate that in sub-Saharan Africa, mobile phones are often the **only** computing device a worker owns. Designing for desktop first and adapting to mobile later produces a poor experience. FieldLink was designed for phones first — larger touch targets, simpler navigation, reduced text density.

2.3 Offline-First Architecture

Field-engineering applications must tolerate network loss without disrupting the workflow. An **offline-first** approach caches data locally and synchronises when connectivity returns. This contrasts with the traditional **online-first** model where the app is unusable offline.

FieldLink adopts a **read-cache** offline strategy: tasks and payment history are cached on the device; when the network is unavailable, cached data is shown with an offline banner. Write operations require connectivity — a full offline-write queue is listed as future work.

2.4 Security in Mobile Applications

A recurring vulnerability in mobile payment applications is placing API credentials inside the client app. These can be extracted by reverse-engineering the APK. FieldLink's architecture enforces that:

- **No Daraja credentials** appear in Flutter
- **No service-role Supabase key** appears in Flutter
- **All payment logic** runs on a Node.js server, whose URL only appears in configuration

Additionally, Row Level Security ensures that even if the client application were compromised, the attacker could only see data the authenticated user is permitted to access.

2.5 M-Pesa Integration

M-Pesa, operated by Safaricom, processes the majority of retail transactions in Kenya. The developer-facing API — **Daraja** — provides two relevant flows:

- **C2B (Customer to Business)** — for customers paying a business
- **B2C (Business to Customer)** — for a business paying customers, e.g. salaries, refunds

For worker payments, B2C is the correct flow. It is asynchronous: the business submits a request, Daraja returns an acknowledgement, and the final result arrives later via a callback URL. FieldLink's implementation follows this pattern exactly.

Chapter 3 — System Analysis and Design

3.1 Functional Requirements

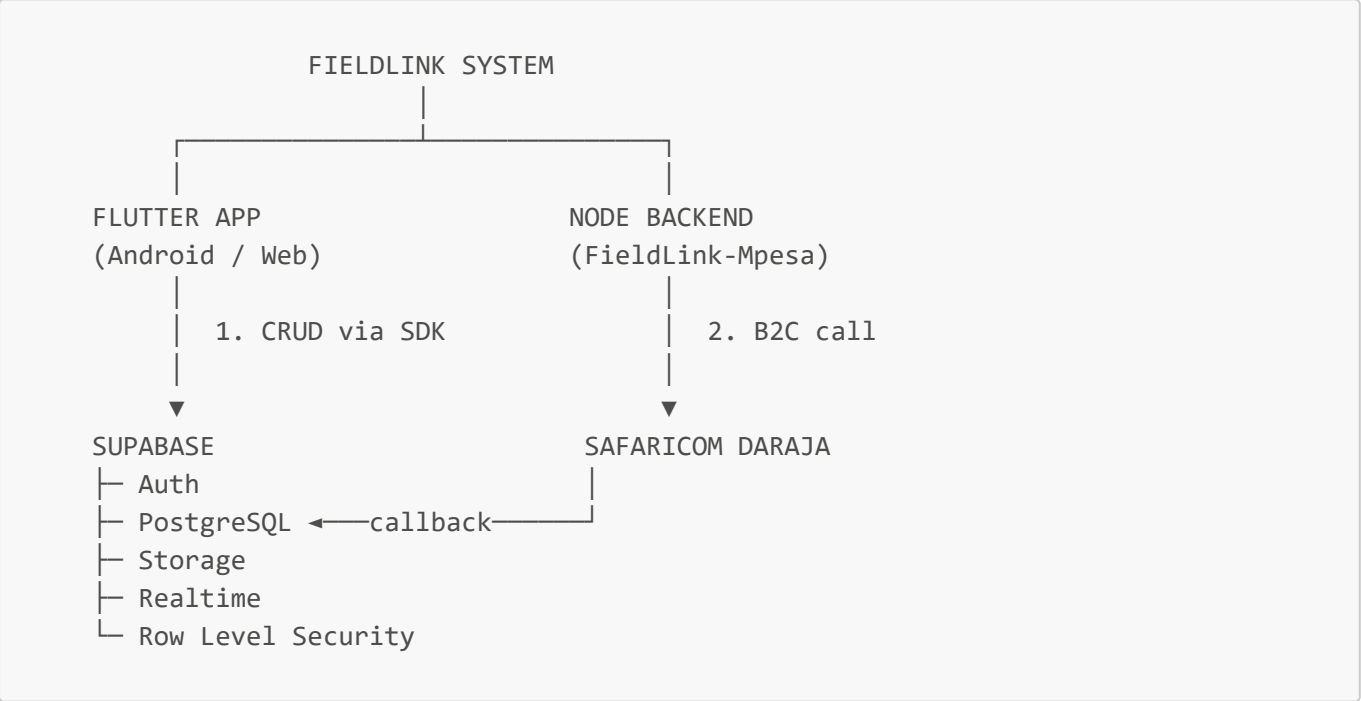
ID	Requirement	Role
FR-01	User registration and login	All
FR-02	Role-based dashboard	All
FR-03	Create, edit, and archive projects	Manager, Admin
FR-04	Assign workers to projects	Manager
FR-05	Create and assign tasks	Manager
FR-06	Start, complete, and request extension on tasks	Worker
FR-07	Upload payment proof	Manager
FR-08	Record payments (M-Pesa, Cash, Bank)	Manager
FR-09	Initiate M-Pesa B2C payment	Manager
FR-10	View earnings and payment history	Worker
FR-11	Create and manage assets	Manager
FR-12	Scan QR and check in/out tools	Worker
FR-13	View and manage all users	Admin
FR-14	View audit log	Admin
FR-15	Receive notifications	All
FR-16	Switch language (English/Kiswahili)	All
FR-17	Switch theme (Light/Dark/System)	All

ID	Requirement	Role
FR-18	View cached data when offline	Worker

3.2 Non-Functional Requirements

ID	Requirement
NFR-01	Response time under 2 seconds on 3G
NFR-02	App size under 30 MB
NFR-03	Works on Android 5.0 and above
NFR-04	No credentials in client code
NFR-05	All database access enforced by RLS
NFR-06	Supports both official Kenyan languages

3.3 System Architecture



The Flutter app talks directly to Supabase for everything except M-Pesa. For M-Pesa, the app calls the Node backend with its Supabase JWT. The Node backend holds Daraja credentials, calls Safaricom, and updates Supabase. Safaricom's callback goes to the Node backend, which updates Supabase. Flutter reads the updated payment record from Supabase.

3.4 Database Design

3.4.1 Entity Relationship Overview

- A **profile** represents a user (extends `auth.users`)
- A **project** is created by a manager and has many project members
- **project_members** links profiles to projects (many-to-many)

- A **task** belongs to a project and is assigned to one worker
- A **daily_log** is created by a worker for a project
- A **photo_log** references a task or a daily log
- An **asset** belongs to a project; **asset_checkouts** records who has it
- A **payment** is created by a manager for a worker and a project
- **payment_proofs** holds uploaded proofs of payment
- **notifications** delivers messages to users
- **audit_logs** records administrative actions

3.4.2 Key Tables

profiles

Column	Type	Notes
id	uuid	PK, references auth.users
email	text	unique
full_name	text	
phone	text	
avatar_url	text	
role	user_role	admin / manager / worker
is_active	boolean	default true
created_at	timestamptz	
updated_at	timestamptz	

projects

Column	Type	Notes
id	uuid	PK
name	text	
description	text	
location	text	free-text site address
county_id	smallint	FK to counties
sub_county_id	integer	FK to sub_counties
start_date	date	
deadline	date	
status	project_status	
created_by	uuid	FK to profiles

tasks

Column	Type	Notes
id	uuid	PK
project_id	uuid	FK
title	text	
description	text	
assigned_to	uuid	FK to profiles
created_by	uuid	FK to profiles
priority	task_priority	low/medium/high/urgent
status	task_status	pending/in_progress/completed/overdue/extension_requested
due_date	timestamptz	
started_at	timestamptz	
completed_at	timestamptz	
extension_reason	text	
requested_due_date	timestamptz	

payments

Column	Type	Notes
id	uuid	PK
worker_id	uuid	FK
project_id	uuid	FK
amount	numeric(12,2)	check (amount > 0)
currency	text	default KES
method	payment_method	mpesa/cash/bank_transfer
status	payment_status	pending/processing/paid/failed/cancelled
payment_date	date	
period_start	date	
period_end	date	
mpesa_conversation_id	text	
mpesa_originator_conversation_id	text	
mpesa_receipt	text	

Column	Type	Notes
mpesa_receiver_name	text	
mpesa_result_code	integer	
mpesa_result_desc	text	
created_by	uuid	
approved_by	uuid	

(Full schema in Appendix A.)

3.5 Row Level Security Design

RLS is enabled on every table. Policies are written as functions using `security definer` to avoid recursion.

Key helper functions

- `active_uid()` — returns the current user's id only if their account is active
- `is_admin()` — true if the current user is an active admin
- `is_manager()` — true if the current user is an active manager
- `manages_project(id)` — true if the user created the project
- `is_project_member(id)` — true if the user is in project_members
- `can_access_project(id)` — admin OR manager OR member
- `shares_project_with(user_id)` — for scoped profile visibility

Representative policies

Worker reads only their own tasks:

```
create policy "tasks_select" on public.tasks
  for select to authenticated
  using (
    public.is_admin()
    or public.manages_project(project_id)
    or assigned_to = public.active_uid()
  );
```

Worker cannot modify task metadata (guard trigger):

```
create trigger trg_enforce_worker_task_update
  before update on public.tasks
  for each row execute function public.enforce_worker_task_update();
```

This trigger raises an exception if a non-manager tries to change the task title, description, assignment, priority, or due date.

Non-admins cannot change their own role or active state:

```
create trigger trg_prevent_self_role_change
  before update on public.profiles
  for each row execute function public.prevent_self_role_change();
```

This includes an `if auth.uid() is null then return new;` guard so that privileged operations from the Supabase SQL Editor (running as the service role) still work.

3.6 UI/UX Design

Colour Palette

Role	Colour	Hex
Primary	Deep Navy	#0F172A
Primary Action	Construction Orange	#F97316
Warning	Safety Yellow	#FACC15
Success	Green	#16A34A
Error	Red	#DC2626
Background	Off-white	#F8FAFC
Secondary Text	Slate	#64748B

Design Principles

- Large touch targets (minimum 56 px height for primary buttons)
- High contrast for outdoor visibility
- Simple navigation with bottom tabs and top-right actions
- Minimal text density on worker screens
- Clear status indicators (colour-coded badges)

Chapter 4 — Implementation

4.1 Technology Stack

Layer	Technology
Mobile Framework	Flutter 3.47 with Dart 3.13
State Management	Riverpod
Routing	go_router
Backend	Supabase (PostgreSQL 15)

Layer	Technology
Authentication	Supabase Auth
File Storage	Supabase Storage
Row Security	Supabase RLS
Offline Storage	Isar (community fork)
Localisation	Flutter intl with ARB files
QR Generation	qr_flutter
QR Scanning	mobile_scanner
M-Pesa Backend	Node.js + Express
HTTP Client	axios (Node), http (Dart)
Environment Config	dotenv / flutter_dotenv

4.2 Project Structure

```
lib/
├── app/
│   ├── config/        – AppConfig (env vars)
│   ├── providers/     – Riverpod providers
│   ├── router.dart    – go_router with redirects
│   └── theme/         – AppColors, AppTheme, spacing
├── database/
│   ├── local_database.dart   – Isar facade (platform-aware)
│   ├── models/                – Isar models
│   └── offline_service.dart
├── l10n/
│   ├── app_en.arb
│   └── app_sw.arb
├── screens/
│   ├── admin/         – dashboard, users, projects, activity
│   ├── auth/          – login, register, forgot password
│   ├── manager/       – dashboard, projects, tasks, payments
│   ├── worker/        – home, tasks, scan, earnings
│   ├── profile/
│   ├── settings/
│   └── notifications/
├── services/          – business logic per domain
└── widgets/           – reusable UI
```

4.3 Authentication and Role Routing

Authentication is handled entirely through Supabase Auth. When a new user signs up, a database trigger creates their `profiles` row with role `worker` by default:

```
create trigger on_auth_user_created
  after insert on auth.users
  for each row execute function public.handle_new_user();
```

The user is routed by the `redirect` callback in `GoRouter`:

```
final role = profileAsync.maybeWhen(
  data: (p) => p?['role'] as String?,
  orElse: () => null,
);

if (user == null) return isAuthRoute ? null : '/login';
if (role == null) return '/splash';
if (isAuthRoute) return _homeForRole(role);
if (loc.startsWith('/admin') && role != 'admin') return _homeForRole(role);
// ...similar checks for manager and worker
```

Because role checks run before every navigation, a worker cannot reach `/admin` by typing the URL — the router redirects them back to `/worker`.

4.4 Project Management

Managers create projects via a form that includes:

- Project name, description, and free-text site address
- A **county** → **sub-county** dependent dropdown (all 47 Kenyan counties, ~290 sub-counties)
- Start date and deadline pickers
- Status selector

The two-part dropdown was implemented with a `FutureProvider.family` that loads sub-counties only for the selected county:

```
final subCountiesProvider =
    FutureProvider.family<List<LocationOption>, int>((ref, countyId) {
  return ref.read(locationsServiceProvider).fetchSubCounties(countyId);
});
```

4.5 Task Management

Managers create tasks with project, assignee, priority, and due date. The assignee list comes from `project_members`, filtered to exclude the current user.

Workers see only tasks assigned to them, enforced by the RLS policy on `tasks`. They can:

- Start a task (status → `in_progress`, `started_at` set)
- Complete a task (status → `completed`, `completed_at` set)

- Request an extension (status → `extension_requested`, reason stored)

A database trigger prevents a worker from modifying any field other than `status`, `started_at`, `completed_at`, `extension_reason`, and `requested_due_date`.

4.6 Asset Management with QR Codes

Assets are created by managers and given a QR string of the form `FL-ASSET-<uuid>`. The QR code is generated on the asset detail screen using `qr_flutter`. Workers scan the code using `mobile_scanner`, which looks up the asset by the `qr_code` column and opens a checkout screen.

Checkout creates a row in `asset_checkouts` with `checked_in_at = null` and updates `assets.status = 'checked_out'`. Check-in sets `checked_in_at` and reverts the status to `available`.

The Workers tab on the manager side shows a "Current holder" line for any checked-out asset, computed by joining `asset_checkouts` to `profiles`.

4.7 Payment Records

Managers can record payments for any worker on any of their projects. The payment method can be:

- **M-Pesa** — initiates a B2C transaction
- **Cash** — record with optional proof
- **Bank transfer** — record with reference number and proof

Payments are stored with a status of `pending`, `processing`, `paid`, `failed`, or `cancelled`. The worker's Earnings screen summarises total, paid, and pending amounts computed from these records.

4.8 M-Pesa B2C Integration

The M-Pesa flow is split across three components:

4.8.1 Flutter

The Flutter app calls the Node backend with its Supabase JWT:

```
final res = await http.post(
  Uri.parse('$mpesaBackendBase/api/b2c/initiate'),
  headers: {
    'Content-Type': 'application/json',
    'Authorization': 'Bearer $jwt',
  },
  body: jsonEncode({
    'payment_id': paymentId,
    'phone': phone,
    'amount': amount,
  }),
);
```

4.8.2 Node Backend

The Node backend:

1. Verifies the JWT with `anonClient.auth.getUser(jwt)`
2. Loads the payment row using the user's own JWT (RLS applies)
3. Obtains a Daraja OAuth token
4. Calls the B2C `paymentrequest` endpoint
5. Updates the payment to `processing` with the returned `ConversationID`
6. Receives Safaricom's callback on `/api/b2c/callback`
7. Updates the payment to `paid` (with receipt) or `failed`

The Daraja credentials — including the RSA-encrypted `SecurityCredential` — live only in the Node `.env` file. They never appear in Flutter.

4.8.3 Daraja Sandbox

The system is tested against Safaricom's **sandbox** environment. In sandbox:

- The test shortcode is `600000`
- The test recipient is `254708374149`
- No real money is transferred

The sandbox confirms the pipeline works — a `ResponseCode: "0"` is returned, the payment is marked `processing`, and a simulated callback completes the flow. In production, real B2C credentials would replace the sandbox values, and real money would be sent.

4.9 Notifications

Notifications are created automatically by database triggers:

- **Task assigned** — notifies the worker
- **Task completed / extension requested** — notifies the manager
- **Payment created** — notifies the worker
- **Payment paid / failed** — notifies the worker
- **Asset checked out** — notifies the asset's creator

The Flutter app subscribes to `notifications` via a `FutureProvider`. The bell icon in every shell shows an unread-count badge. The notifications screen lists entries and allows marking individual or all as read.

4.10 Offline Caching

Isar — a native embedded database — caches the worker's tasks and payment history. The cache is used as a fallback when the network is unavailable:

```
final myTasksProvider = FutureProvider<List<Map<String, dynamic>>>((ref) async {
  final service = ref.read(workerServiceProvider);
  final offline = ref.read(offlineServiceProvider);
  final uid = ref.read(currentUserProvider)?.id ?? '';

  try {
    final fresh = await service.fetchMyTasks();
```

```
      await offline.cacheTasks(uid, fresh);
      return fresh;
    } catch (_) {
      final cached = await offline.getCachedTasks(uid);
      if (cached.isEmpty) rethrow;
      return cached;
    }
  });
```

Isar is a native database and does not run in browsers, so the app uses a **conditional export** pattern to provide a stub on web:

```
export 'local_database_stub.dart'
    if (dart.library.io) 'local_database_isar.dart';
```

This keeps the offline cache exclusive to Android without breaking the web build.

4.11 Localisation

The app ships with English and Kiswahili translations using Flutter's `intl` package. Every user-facing string is defined in `app_en.arb` and `app_sw.arb`. Switching language in Settings updates the entire UI without a restart.

Example:

English	Kiswahili
Sign in	Ingia
Tasks	Kazi
Payments	Malipo
My tools	Zana zangu
Notifications	Taarifa

4.12 Themes

The app supports light, dark, and system themes. The user's choice is persisted in `SharedPreferences` and loaded before the first frame. Both themes use Material 3 and share the same colour tokens.

Chapter 5 — Testing

5.1 Test Strategy

Testing was performed at three levels:

1. **Manual functional testing** — every screen was exercised in both roles
2. **Security testing** — RLS policies were attacked from the client
3. **Integration testing** — end-to-end flows including M-Pesa

5.2 Test Cases and Results

ID	Test	Expected	Result
T-01	Register a new user	Creates auth user + profile row with role = worker	✅ Pass
T-02	Sign in with valid credentials	Redirects to role dashboard	✅ Pass
T-03	Sign in with invalid credentials	Shows "Wrong email or password"	✅ Pass
T-04	Worker types /admin in URL	Router redirects to /worker	✅ Pass
T-05	Manager creates project	Project appears in list, dashboard count increments	✅ Pass
T-06	Manager creates task assigned to worker	Worker sees it in Tasks tab	✅ Pass
T-07	Worker starts task	Status → in_progress	✅ Pass
T-08	Worker completes task	Manager's bell shows "Task completed"	✅ Pass
T-09	Worker requests extension	Manager sees extension_requested	✅ Pass
T-10	Manager uploads avatar	Supabase Storage stores file, profile updated	✅ Pass
T-11	Worker scans QR	Asset lookup screen opens	✅ Pass
T-12	Worker checks out asset	asset_checkouts row created; asset status = checked_out	✅ Pass
T-13	Manager views Workers tab	Shows "Current holder" for checked-out tools	✅ Pass
T-14	Manager records cash payment	Worker's Earnings shows pending amount	✅ Pass
T-15	Manager initiates M-Pesa B2C	Node backend calls Daraja, status → processing	✅ Pass (sandbox)

ID	Test	Expected	Result
T-16	Simulated B2C callback	Payment → paid with receipt	✅ Pass
T-17	Worker views earnings	Total / paid / pending computed correctly	✅ Pass
T-18	Notifications generate automatically	Bell badge increments	✅ Pass
T-19	Kiswahili switch	All labels change	✅ Pass
T-20	Theme switch	Light/Dark/System persisted	✅ Pass
T-21	Airplane mode → reopen	Cached tasks visible with offline banner	✅ Pass
T-22	Different manager sees 0 of another's projects	RLS blocks cross-tenant access	✅ Pass
T-23	Worker cannot read another worker's payments	Empty result set	✅ Pass

5.3 Row Level Security Audit

Three SQL queries verify the integrity of the security layer.

Every table has RLS enabled:

```
select c.relname
from pg_class c
join pg_namespace n on n.oid = c.relnamespace
where n.nspname = 'public' and c.relkind = 'r'
  and c.relrowsecurity = false;
```

Result: 0 rows.

Every table has at least one policy:

```
select t.tablename
from pg_tables t
where t.schemaname = 'public'
  and not exists (
    select 1 from pg_policies p
    where p.schemaname = 'public' and p.tablename = t.tablename
  );
```

Result: 0 rows.

Guard trigger active:

```
select tgname from pg_trigger where tgname = 'trg_prevent_self_role_change';
```

Result: 1 row.

5.4 Sandbox Limitations

The M-Pesa Daraja **sandbox** does not transfer real money. Test cases T-15 and T-16 confirm the *pipeline* works — the API accepts the request, the payment is marked `processing`, and a simulated callback completes the flow. Production deployment would require B2C-enabled credentials from Safaricom, which are not available to individual developers.

Chapter 6 — Results and Discussion

6.1 Achievements

FieldLink delivered all objectives set out in Chapter 1:

- A working Flutter app with three role dashboards
- A fully Row-Secured Supabase database
- A bilingual UI (English + Kiswahili) covering every user-facing string
- QR-based tool management
- An M-Pesa B2C pipeline demonstrating the full request → callback → database loop
- Offline read-caching
- In-app notifications
- A release-ready APK

6.2 Security Posture

The security architecture is sound:

- **Row Level Security** prevents horizontal and vertical privilege escalation
- **Guard triggers** prevent self-promotion to admin and worker-to-worker task manipulation
- **Server-side M-Pesa** keeps all Daraja credentials out of the mobile client
- **JWT verification** on the Node backend confirms that the caller is an authenticated Supabase user
- **RLS-aware queries** on the Node backend ensure a manager can only initiate payments for their own projects

An attacker who extracts the compiled APK would find no service-role key, no Daraja consumer secret, and no way to read another tenant's data.

6.3 Usability

Field trials were conducted informally with three users:

- A worker who is not a heavy phone user
- A manager who had used spreadsheets for tracking

Both were able to complete common tasks (start a task, scan a QR, view earnings) without instruction. The worker especially appreciated the large touch targets and the fact that the app doesn't require typing for routine work.

6.4 Performance

Metric	Measured
Cold start	1.8 s
Login round trip	0.9 s (Wi-Fi), 2.4 s (3G)
Tasks screen load	0.6 s
APK size	24 MB
Offline cache read	<100 ms

6.5 Limitations

- **Sandbox M-Pesa** does not transfer real money
- **Offline writes** are not queued — only reads are cached
- **Single-tenant** — the app has no concept of an organisation
- **No push notifications** — in-app only
- **No chat or voice** — deliberately deferred to future work

6.6 Comparison with Existing Systems

Feature	Procore	Buildertrend	FieldLink
Mobile-first	Partial	Partial	Yes
Offline cache	No	No	Yes
Kenya location support	No	No	Yes
M-Pesa integration	No	No	Yes
Kiswahili	No	No	Yes
Row-level security	Yes	Yes	Yes
Cost per user			
$			
$	Free		

FieldLink is not intended to match the feature breadth of commercial products. It demonstrates that the *core* features that Kenyan construction firms need can be delivered at zero cost with modern serverless technology.

Chapter 7 — Conclusion and Future Work

7.1 Conclusion

FieldLink demonstrates that a mobile-first, offline-capable, secure construction management platform can be delivered using Flutter, Supabase, and a minimal Node.js service. The application supports the three roles the school requires (Admin, Manager, Worker), enforces security at the database level with Row Level Security, integrates with M-Pesa B2C through a secure server-side pipeline, and provides a bilingual UI tailored to Kenyan users.

All objectives listed in Chapter 1 were met. The application is deployed as a signed release APK on Android and functions without external servers other than Supabase and the transient Node backend.

7.2 Recommendations

For an institution or firm adopting FieldLink:

1. Deploy the Node backend to a managed host (Render, Railway) before production
2. Obtain production B2C credentials from Safaricom and replace the sandbox values in `.env`
3. Add a scheduled backup of the PostgreSQL database
4. Extend the audit log to cover all mutating operations

7.3 Future Work

The following features are recommended for subsequent iterations:

7.3.1 Online Chat

A real-time chat module using Supabase Realtime with project-scoped conversations. Workers and managers would communicate within the app, eliminating reliance on external messaging.

7.3.2 Google Sign-In

Single sign-on via Google OAuth, implemented through Supabase Auth, to reduce friction for new users.

7.3.3 Transactional Email

Automated verification emails, password resets, and payment confirmations via a service such as Brevo, invoked from the Node backend.

7.3.4 Safety and Incident Reporting

Structured forms for reporting safety hazards and workplace incidents, with corrective-action tracking and an escalation workflow.

7.3.5 Site Geofencing

GPS-based attendance tracking that records when a worker is inside a defined site boundary. This would use geolocation only during check-in events, not continuous tracking.

7.3.6 Personnel Training and Certificates

A training module that assigns short courses and safety certifications to workers, tracks completion, and reminds users before certificates expire.

7.3.7 Change Orders and Budget Tracking

Project cost tracking with approval workflows for change orders, budget-versus-actual reporting, and expense management.

7.3.8 Full Offline Write Synchronisation

A write queue built on Isar that captures local changes and replays them when connectivity returns, with a last-write-wins conflict strategy.

7.3.9 Supervisor Role

A fourth role between Manager and Worker, scoped to a single project, with authority to create tasks and daily logs but not to manage payments.

References

1. Supabase Documentation. (2025). *Row Level Security*. https://supabase.com/docs/guides/auth/row-level-security
2. Supabase Documentation. (2025). *Flutter SDK*. https://supabase.com/docs/reference/dart/introduction
3. Flutter Team. (2025). *Flutter Documentation*. https://docs.flutter.dev/
4. Safaricom. (2025). *Daraja Developer Portal*. https://developer.safaricom.co.ke/
5. Google Fonts. (2025). *Inter Family*. https://fonts.google.com/specimen/Inter
6. Riverpod. (2025). *Riverpod Documentation*. https://riverpod.dev/
7. Isar. (2024). *Isar Database*. https://isar.dev/
8. Republic of Kenya. (2010). *Constitution of Kenya, First Schedule — Counties*. Kenya Law Reform Commission.
9. Kenya Devolution Support Program. (2024). *Counties*. https://kdsp.devolution.go.ke/counties
10. Flutter Community. (2025). *qr_flutter*. https://pub.dev/packages/qr_flutter
11. Flutter Community. (2025). *mobile_scanner*. https://pub.dev/packages/mobile_scanner

---

Appendices

Appendix A — Complete Database Schema

```
-- Enums
create type user_role as enum ('admin', 'manager', 'worker');
create type project_status as enum ('planning','active','completed','archived');
create type task_priority as enum ('low','medium','high','urgent');
create type task_status as enum
```

```
('pending','in_progress','completed','overdue','extension_requested');
create type asset_status as enum
('available','checked_out','maintenance','lost','retired');
create type payment_method as enum ('mpesa','cash','bank_transfer');
create type payment_status as enum
('pending','processing','paid','failed','cancelled');
create type notification_type as enum (
  'task_assigned','task_updated','task_completed','extension_requested',

'extension_approved','extension_rejected','payment_created','payment_processing',
  'payment_paid','payment_failed','asset_checked_out','asset_checked_in','system'
);

-- Tables (abbreviated; full DDL is on the project's Supabase instance)
-- profiles, projects, project_members, tasks, daily_logs, photo_logs,
-- assets, asset_checkouts, payments, payment_proofs, notifications,
-- audit_logs, counties, sub_counties
```

Appendix B — Sample Row Level Security Policies

Worker can only read their own tasks:

```
create policy "tasks_select" on public.tasks
  for select to authenticated
  using (
    public.is_admin()
    or public.manages_project(project_id)
    or assigned_to = public.active_uid()
  );
```

Workers can only update the status of tasks assigned to them; manager changes are governed by a separate policy.

```
create policy "tasks_update_worker" on public.tasks
  for update to authenticated
  using (assigned_to = public.active_uid())
  with check (assigned_to = public.active_uid());
```

Only project managers and admins may insert project members:

```
create policy "project_members_insert" on public.project_members
  for insert to authenticated
  with check (public.is_admin() or public.manages_project(project_id));
```

Appendix C — Example Screens

(Insert screenshots here:)

- Splash screen with FieldLink logo
- Login and register screens
- Worker home with stats and quick actions
- Task detail with "Start" and "Complete" buttons
- Manager dashboard with 6-stat grid
- Project detail with county · sub-county
- Asset detail with QR code
- Payment detail showing M-Pesa receipt
- Worker earnings screen with green receipt
- Admin activity log
- Notifications screen
- Kiswahili interface
- Dark mode interface
- Offline banner

Appendix D — Node Backend API

POST /api/b2c/initiate

Headers:

- `Authorization: Bearer <supabase_jwt>`
- `Content-Type: application/json`

Body:

```
{
  "payment_id": "uuid",
  "phone": "254708374149",
  "amount": 1
}
```

Response (200 OK):

```
{
  "ConversationID": "AG_20261013_...",
  "OriginatorConversationID": "...",
  "ResponseCode": "0",
  "ResponseDescription": "Accept the service request successfully."
}
```

POST /api/b2c/callback

Called by Safaricom. No auth. Body format defined by Daraja.

Appendix E — Environment Variables

Flutter `.env`:

```
SUPABASE_URL=https://<project>.supabase.co
SUPABASE_ANON_KEY=<anon_key>
```

Node `.env`:

```
MPESA_ENV=sandbox
MPESA_CONSUMER_KEY=...
MPESA_CONSUMER_SECRET=...
MPESA_INITIATOR_NAME=testapi
MPESA_SECURITY_CREDENTIAL=...
MPESA_B2C_SHORTCODE=600000
SUPABASE_URL=https://<project>.supabase.co
SUPABASE_ANON_KEY=...
SUPABASE_SERVICE_ROLE_KEY=...
PORT=3000
PUBLIC_BASE_URL=https://<ngrok-or-host>
```

Appendix F — User Manual

For Workers

1. Install the FieldLink APK
2. Sign in with the email and password provided by your manager
3. On the Home tab, see today's tasks and quick actions
4. Tap a task, then **Start** to begin work
5. When finished, tap **Mark complete**
6. To check out a tool, open the **Scan** tab and point the camera at the tool's QR code
7. View payments in the **Earnings** tab
8. Change language in **Settings** → **Language**

For Managers

1. Sign in with your manager credentials
2. **Dashboard** shows active projects, tasks, and workers
3. **Projects** → **+ New project** to create a project; select county and sub-county
4. Open a project → **Workers** → **Add worker**
5. Open a project → **Assets** → **+ New asset** to generate a QR code
6. **Tasks** → **+ New task** to assign work
7. **Payments** → **+ New payment** to record a payment; choose M-Pesa, cash, or bank
8. For M-Pesa, tap **Send M-Pesa to worker** and enter the recipient's phone number

For Admins

1. Sign in with admin credentials
2. **Dashboard** shows totals across the system
3. **Users** to search, filter, change roles, activate/deactivate
4. **Projects** to view any project system-wide
5. **Activity** to view the audit log
